Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Machine Learning for Genomics
Autumn Semester 2026

Project 1: Prediction of Gene Expression from Chromatin Landscape

Assigned on: **4:00pm on 07.10.2026** Due by: **11:59am on 28.10.2026**

Overview

This is the first graded project of the course. Its topic is to use chromatin information to predict gene expression. The goal is to build a simple (or more sophisticated) machine learning model that uses the available chromatin tracks to predict the expression of genes in an **unseen cell line**. The better your predictions agree with the measured expression, the better your model has captured the regulatory logic that links chromatin state to transcription.

In particular, in this project you will:

a) explore chromatin (a.k.a. epigenetic) datasets, such as histone modifications and chromatin accessibility,

b) choose and implement a prediction model that takes these datasets as input and predicts gene expression,

c) evaluate the performance of your method and submit predictions for the unseen cell line on Kaggle,

d) submit your code and a brief summary of your solution on Moodle.

We have split the work into several work packages. You do not need to follow this division exactly, but it indicates which steps we consider important. The submission workflow (Kaggle competition, team registration, submission limit, final submission selection and Moodle hand-in) is the same as in Project 0.

A submission consists of two parts: your selected predictions on Kaggle and your code and solution summary on Moodle (Work Package 1.4). A submission is only complete if both parts are handed in before the deadline.

Data

Transcription factor binding, histone modifications and chromatin accessibility all play a role in the up- or down-regulation of nearby genes. In this project, you will use genome-wide profiles of histone modifications and/or chromatin accessibility to predict gene expression. To test the robustness of your model, you will work with three different human cell lines, X_1, X_2 and X_3. **All files correspond to the hg38/GRCh38 version of the human genome.** For each of the three cell lines, the following data are available:

- Chromatin accessibility (DNase-seq)
- H3K4me3 (ChIP-seq)

- H3K4me1 (ChIP-seq)
- H3K27ac (ChIP-seq)
- H3K36me3 (ChIP-seq)
- H3K27me3 (ChIP-seq)
- H3K9me3 (ChIP-seq)

In addition, the following are provided:

- Gene information (obtained using CAGE and RefSeq): gene name, chromosome, gene body coordinates, transcription start site (TSS) coordinates and strand, for all three cell lines.
- Gene expression (obtained using CAGE): gene name and expression value, for X_1 and X_2 only.

Chromosome split. Let chr denote the set of all autosomes (non-sex chromosomes). We define A, B and C as disjoint subsets of chr such that $A \cup B \cup C = chr$:

- $A = \{$chr2, chr3, chr4, chr5, chr6, chr7, chr8, chr9, chr10, chr11, chr12, chr13, chr15, chr16, chr17, chr18, chr20, chr21, chr22$\}$
- $B = \{$chr14, chr19$\}$
- $C = \{$chr1$\}$

You are expected to train your model on set A of cell lines X_1 and X_2 and to validate it on set B of X_1 and X_2. You are free to use different splits of $A \cup B$ for training and validation. Your model will be tested on the genes of set C of cell line X_3, whose expression values are hidden.

Normalization. To make the features and targets comparable across cell lines, all datasets have already been normalized using the ChIPIN method (Polovinkin et al., *BMC Bioinformatics*, 2020), which is outside the scope of this project.

File formats. Each feature dataset (ChIP-seq and DNase-seq) is provided in two formats, and you are free to use either or both:

- **BED narrowPeak** (`.bed`): called peaks, i.e. discrete regions of enriched signal. The files follow the standard 10-column narrowPeak format without a header.
- **BigWig** (`.bw` / `.bigwig`): genome-wide continuous signal, which can be queried over arbitrary intervals (e.g. with `pyBigWig`).

Details on these formats can be found at https://genome.ucsc.edu/FAQ/FAQformat.html. The feature tracks cover all chromosomes, including the sex chromosomes; the genes are restricted to autosomes. You are encouraged to visualize the data using the Integrative Genomics Viewer (IGV) or the UCSC Genome Browser.

Files. The data are provided in the Kaggle competition as a dataset, with the following archives. Each feature archive contains one file per cell line (X_1, X_2, X_3).

- `CAGE-train.zip`: gene information and expression targets: `X{1,2}_train_info.tsv` and `X{1,2}_train_y.tsv` (set A), `X{1,2}_val_info.tsv` and `X{1,2}_val_y.tsv` (set B), and `X3_test_info.tsv`, which lists the test genes (set C of X_3).
- `<assay>-bed.zip` and `<assay>-bigwig.zip` for each of the seven assays (DNase, H3K4me3, H3K4me1, H3K27ac, H3K36me3, H3K27me3, H3K9me3).
- `sample_submission.zip`: an example submission showing the exact required format.

A copy of `sample_submission.csv` is also available on the competition's Data tab.

Work Package 1.0 – Setup

a) Join the Project 1 competition using the invitation link . Read and accept the competition rules.

b) Form your team on Kaggle (Team tab of the competition). Teams are specific to each project, so you need to form your team again, not necessarily in the same composition as registered for Project 0. Teams have at most 2 members (on Kaggle it is 3 to allow for exceptional situations; the limit of 2 applies nevertheless). Merge teams before making submissions.

c) **register your team on Moodle by 21.10.2026.** Every student must complete the *Team registration* activity individually, stating their Kaggle username (as shown in their profile URL), their Kaggle team name, and the names of their teammates. Kaggle accounts that cannot be matched to a Moodle registration will not be graded.

d) We provide a starter notebook on Moodle and Kaggle. To view the Kaggle version (with the data attached), join the following group on Kaggle. The notebook shows how to locate and load the gene information, the expression targets and the chromatin tracks, and how to write a correctly formatted `submission.csv`. It contains no model: it submits a constant placeholder prediction, which scores 0, and leaves a marked section for your solution.

Work Package 1.1 – Modelling Choices & Data Pre-processing

Using the data described above, you will need to make some important choices:

a) Which histone marks should I use?

b) Should I use the chromatin accessibility data?

c) Should I use the `.bed` files, the `.bigwig` files, or a combination of both?

d) Should I use the signal only from specific genomic regions (promoter, gene body, distal regions)?

e) Should the signal be further transformed or normalized?

f) Should I use the DNA sequence (reference genome) as an additional input? These data are not provided, but the sequence of any set of regions can easily be extracted from the hg38 reference with `bedtools getfasta` (or its Python wrapper `pybedtools`).

The goal of this work package is to decide which pre-processing steps answer these questions best, so that you can achieve the best result in the next work package. A central question is which part of the genome is informative about the expression of a given gene. For perspective: distal regulatory elements (enhancers)

up to 1 million base pairs away from a gene's TSS can contribute to its regulation, but the probability of such events is low. The likelihood that a DNA segment affects the regulation of a gene decreases with its distance from the gene: about 47% of the information is encoded within ± 20 kb of the TSS, and about 84% within ± 100 kb. We expect you to determine systematically which features are the best input for your model. Visualizing the data in IGV or the UCSC Genome Browser can help answer some of the questions above.

Do not hesitate to search the web and read papers of the ENCODE consortium on this topic.

Work Package 1.2 – Model Building

You are now ready to build a model that predicts gene expression from chromatin information. But which model should you use? A linear model, a non-linear model (e.g. decision trees or gradient boosting), or something more complex? To answer these questions, you are encouraged to search the literature on this topic.

Train and validate your model on sets A and B of cell lines X_1 and X_2. Keep in mind that the test set comes from a cell line your model has never seen: a model that performs well across chromosomes of the same cell line does not necessarily generalize across cell lines. Validation schemes that hold out a cell line (e.g. training on X_1 and validating on X_2) help you check this. Use your local validation to compare models before submitting: you have a limited number of submissions (see below), so do not use the leaderboard as your validation set.

Work Package 1.3 – Prediction and Kaggle Submission

Using the model selected in Work Package 1.2, predict the expression of all test genes (set C of X_3, listed in `X3_test_info.tsv`). Write your predictions to a CSV file with the header `gene_name,gex_predicted` and exactly one row per test gene, **without an index column** (in pandas: `to_csv(path, index=False)`). The starter notebook contains format checks that catch the most common mistakes.

Submit the file either directly from your Kaggle notebook's output or by uploading it on the competition page, and verify that it receives a score on the leaderboard. Before the deadline, select the submission you want to be graded (see Submission Rules).

Work Package 1.4 – Code and Solution Summary on Moodle (required)

By the deadline, **every student** must individually upload a single `.zip` file to the *Project 1 submission* activity on Moodle. Teammates may upload the same code, but the contribution statement (see below) must be your own. The archive must contain:

a) Code. The notebook(s) or scripts that produce your *selected* Kaggle submission from the provided data. Running your code must reproduce the selected `submission.csv`; fix all random seeds so that the result is deterministic.

b) Environment. A conda `environment.yml` or `requirements.txt` listing all packages you used. If you worked entirely in a Kaggle notebook, list the packages you installed in addition to the Kaggle default environment (e.g. `pyBigWig`), or state in the README that you installed none.

c) README. Brief instructions on how to run your code, including where the data are expected to be located and any additional files you downloaded (e.g. the reference genome).

d) Solution summary (at most one page, PDF or Markdown) containing:

- your Kaggle team name,

- the timestamp and public score of your selected Kaggle submission,
- a brief description of your approach (features and pre-processing, model, how you chose them) and your local validation scores,
- a short statement of which parts of the work you did yourself.

Evaluation Metric

Since the end goal is to identify highly and lowly expressed genes, submissions are evaluated using **Spearman's rank correlation coefficient** ρ between the predicted and the CAGE-measured expression of the test genes; higher is better. Spearman's ρ is the Pearson correlation between the ranks of the two quantities. Let f_i and y_i be the predicted and true expression of gene i, $R(\cdot)$ the rank, and $\bar{R}_f$, $\bar{R}_y$ the mean ranks. Then

$$\rho = \frac{\sum_i \left(R(f_i) - \bar{R}_f\right)\left(R(y_i) - \bar{R}_y\right)}{\sqrt{\sum_i \left(R(f_i) - \bar{R}_f\right)^2 \sum_i \left(R(y_i) - \bar{R}_y\right)^2}}, \tag{1}$$

where tied values receive the average of their ranks. If there are no ties, this reduces to

$$\rho = 1 - \frac{6\sum_i d_i^2}{n(n^2-1)}, \tag{2}$$

where d_i is the difference between the two ranks of gene i and n is the number of genes.

Spearman's ρ depends only on the *ranking* of your predictions, not on their raw values: any strictly increasing transformation of your predictions leaves the score unchanged. Constant predictions have no defined rank correlation and receive a score of 0. You can also use other metrics, such as Pearson's correlation or R^2, during development, but they are not used for grading. The public leaderboard is computed on all test genes and is visible throughout the competition. There is no private leaderboard.

Submission Rules

- **Format.** A CSV file with the header `gene_name,gex_predicted` and exactly one row per gene in `X3_test_info.tsv`, with numeric predictions and no missing values, as in `sample_submission.csv`.
- **Submission limit.** At most **10 submissions per team in total**. Kaggle will not block further submissions, but only your team's first 10 submissions, in chronological order, are eligible for grading and the final ranking; later submissions are ignored regardless of their score. Submissions rejected by Kaggle as errors do not count towards this limit. For teams formed by merging, all submissions made by any member before the merge count towards the team's total.
- **Final submission.** Before the deadline, each team must select **exactly one** submission, among its first 10, to be graded. If no eligible submission is selected, we use the team's best-scoring eligible submission.
- **Code and summary.** Each student must upload their code and solution summary on Moodle as described in Work Package 1.4. Teams whose selected submission is not accompanied by code on Moodle, or cannot be reproduced from that code, are not graded.
- **Data and external resources.** The provided data may only be used for this course. You may use any method or library, publicly available genome annotations, the hg38 reference sequence, and pretrained sequence models as feature extractors. You may **not** attempt to identify the anonymized cell lines, use any external gene expression measurements for them, or use cell-type-specific expression predictions of pretrained models (e.g. the CAGE or RNA output tracks of Enformer or Borzoi).
- **Deadline.** 11:59am on 28.10.2026 (Zurich time), for both the Kaggle and the Moodle parts. Submissions after the deadline are not accepted.

- **Conduct.** One Kaggle account per person. Exchanging ideas within and across teams is encouraged, preferably on the Moodle forum so that everyone benefits. Using multiple accounts, circumventing the submission limit, or privately sharing code or predictions between teams is not allowed. Violations may result in failing the project.

The full competition rules are available on the Rules tab of the competition page. For questions, please use the appropriate Moodle forum, so that everyone benefits from the answer.

Grading

To pass Project 1, your selected submission must reach a Spearman correlation of at least **0.65796** on the leaderboard, and your code and solution summary must be handed in on Moodle as described above.

A bonus of +0.25 on the semester grade is given to the 10% of students who perform best in terms of Spearman's correlation on the leaderboard.